

LAB ACTIVITY: FOUNDATIONS OF LINUX FIREWALLS (iptables & nftables)

```
File Actions Edit View Help
$ sudo nft add rule inet my_filter input tcp dport '{ 22, 80, 443 }' accept
(ekfm2025@kali)-[~/Desktop]
$ sudo iptables -L -n -v
Chain INPUT (policy DROP 24 packets, 3932 bytes)
pkts bytes target prot opt in out source destination
0 0 ACCEPT all -- lo * 0.0.0.0/0 0.0.0.0/0
4229 14M ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0 ctstate RELATED,ESTABLISHED
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:22
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:443

Chain FORWARD (policy DROP 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 3955 packets, 411K bytes)
pkts bytes target prot opt in out source destination

(ekfm2025@kali)-[~/Desktop]
$ sudo nft list ruleset
# Warning: table ip filter is managed by iptables-nft, do not touch!
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy drop;
        iifname "lo" counter packets 0 bytes 0 accept
        ct state related,established counter packets 4231 bytes 13686255 accept
        tcp dport 22 counter packets 0 bytes 0 accept
        tcp dport 80 counter packets 0 bytes 0 accept
        tcp dport 443 counter packets 0 bytes 0 accept
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
table inet my_filter {
    chain input {
        type filter hook input priority filter; policy drop;
        iif "lo" accept
        ct state established,related accept
        tcp dport { 22, 80, 443 } accept
    }
}
```

1. Stateful vs. Stateless: Why did we need the ESTABLISHED,RELATED rule? What would happen to your `sudo apt update` command if that rule were missing, even if OUTPUT is set to ACCEPT?

- The ESTABLISHED, RELATED rule is important because a stateful firewall needs to know about and allow incoming packets that are part of an existing, valid connection that already initiated. Without this rule, `sudo apt update` command would fail because the firewall would see the repository's response as unauthorized "unsolicited" traffic and block it. Setting OUTPUT to ACCEPT would allow the server to send the request out, but it would not be able to get the data back that it needed to complete the update process.

2. Order of Operations: If you placed a DROP ALL rule at the very top of your INPUT chain, would your SSH rule at the bottom still work? Explain why.

- No, SSH rule would not work because iptables operates on a "First Match Wins" logic. Once a packet matches a rule, the target action is applied immediately, and the system stops evaluating any further rules in that chain. Therefore, placing a "DROP ALL" rule at the top would cause the firewall to discard the connection attempt before it ever reaches the specific rule at the bottom allowing SSH traffic.

3. nftables Advantages: Identify one syntax difference between iptables and nftables that makes the latter easier to manage for large-scale networks.

- One syntax difference is that nftables allows to consolidate rules by grouping multiple ports in a single line of logic with curly braces. With iptables, you need three separate rules to permit traffic for SSH, HTTP, and HTTPS. Your nftables ruleset does this in one rule: `tcp dport { 22, 80, 443 } accept`. This simplification results in the configuration being smaller overall, and thus much easier to manage and audit in large networks.